CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 01
Advanced Algorithm Analysis, Correctness & Lower Bounds

What exactly are we claiming when we say an algorithm is "efficient" and "correct" — and how do we prove that no algorithm can do better?

Saturday 26 September 2026 · 3 hours

How CS612 works

- **14 lectures × 3 h**, every Saturday, 26 Sep – 26 Dec
- Each lecture: motivating problem → **Block A** technique → **Block B** proof / deep case → **Block C** design, critique or experiment → synthesis
- Undergraduate algorithms are **assumed**: refreshers R1–R7 are self-study
- You will be asked to commit to answers in class. Being wrong here costs nothing.

Coursework (40)	Marks
Midterm (Sat 14 Nov, L01–L07)	10
Problem sets PS1–PS4 (2.5 each)	10
Algorithm engineering project (team)	15
Individual viva / presentation	5
Final exam (written, no MCQ)	**60**

> **THE COURSE IN ONE SENTENCE**
>
> Given a problem, its assumptions and its constraints: **design**, **justify**, **analyse**, **compare**, **implement** and **experimentally evaluate** an appropriate algorithmic solution.

The questions we will ask every week

Correctness

Why does it work?
Can we **prove** it?
What are the edge cases?

Guarantee

Worst-case? Expected? Amortized?
With high probability?
Approximate?

Assumptions

Model of computation, input distribution, adversary, weights, key universe…
What changes if one is dropped?

Alternatives

What else solves it?
Under which conditions does each one win?

Limits

Is there a **lower bound**?
Is an exact efficient algorithm even plausible?

Evidence

What would an experiment show?
What would it **not** show?

Motivating problem: believe the vendor?

Three sales pitches for a sorting library, for records with 64-bit keys:

1. "Our comparison sort needs only $3n$ comparisons on *any* input."
2. "We sort 10^8 records in $O(n)$ time."
3. "Our sort is $O(n \log n)$, so it is always faster than your $O(n^2)$ insertion sort."

For each: **impossible**, **possible under assumptions** (which?), or **not a meaningful claim**?

PAIRS · 4 MIN

Decide for each claim. Write down the *one assumption* that decides it.

We'll come back to all three by the end of today.

BLOCK A · 00:15–01:00

What exactly are we claiming?

Models · input size · kinds of guarantee · bounds · asymptotics vs. practice

A running time is only meaningful inside a model

Model	Unit-cost operations	Typical use
Word-RAM (word size $w \geq \log_2 n$)	arithmetic, comparison, indexing, bit operations on w-bit words	default for this course; integer sorting, hashing
Comparison model	only $a_i \leq a_j$? queries on keys; everything else free	lower bounds for sorting, searching, selection
Real RAM	exact arithmetic on real numbers	computational geometry
External memory (block size B, memory M)	cost = number of block transfers	databases, huge data

> A lower bound proved in one model says **nothing** about an algorithm that uses operations outside that model.

What is the input size?

Input size = the number of **bits** (or words) needed to write the input down.

Problem	Natural size
Sort n keys of w bits	n words (nw bits)
Graph problems	$n = \|V\|$, $m = \|E\|$ (two parameters)
Is N prime?	$b = \lceil \log_2(N+1) \rceil$ bits — **not** N
0/1 knapsack, capacity W	n items + $\log_2 W$ bits for W

ASK

Trial division tests primality with $\sqrt{N}$ divisions. Polynomial or exponential?

$\sqrt{N} = 2^{b/2}$: **exponential** in the input size. The same trap returns in L07: the $O(nW)$ knapsack DP is *pseudo-polynomial*.

Four different guarantees (do not mix them up)

$T(x)$ = cost on input x; $T(x, r)$ = cost on input x with random bits r.

Guarantee	Definition	Randomness over…
Worst case	$T_{\text{wc}}(n) = \max_{\lvert x\rvert = n} T(x)$	none
Average case	$T_{\text{avg}}(n) = \mathbb{E}_{x \sim \mathcal{D}_n}[T(x)]$	the *input* (assumed distribution $\mathcal{D}_n$)
Expected (randomized alg.)	$T_{\text{exp}}(n) = \max_{\lvert x\rvert = n} \mathbb{E}_r[T(x, r)]$	the algorithm's own coins; input is **worst case**
Amortized	$\max_{\text{sequences } \sigma \text{ of } m \text{ ops}} \frac{1}{m} \sum_i c_i$	none: worst case over *sequences*

> Average case trusts the input distribution. Expected time trusts only your own random number generator. Amortized involves **no probability at all**.

Asymptotic notation, formally

For $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$:

$f = O(g)$	$\exists\, c > 0,\ n_0 :\quad f(n) \leq c\, g(n)\ \text{ for all } n \geq n_0$	upper bound
$f = \Omega(g)$	$\exists\, c > 0,\ n_0 :\quad f(n) \geq c\, g(n)\ \text{ for all } n \geq n_0$	lower bound
$f = \Theta(g)$	$f = O(g)$ and $f = \Omega(g)$	tight bound
$f = o(g)$	$\forall\, c > 0\ \exists\, n_0 :\quad f(n) < c\, g(n)\ \text{ for all } n \geq n_0$	strictly smaller order
$f = \omega(g)$	$g = o(f)$	strictly larger order

THESE ARE STATEMENTS ABOUT FUNCTIONS

O, Ω, Θ compare **two functions**. They are not "cases".

"Worst case" and "best case" choose **which function** we are describing.

Each combination is meaningful: the worst case can have an Ω bound and the best case can have an O bound.

Algorithm bounds vs. problem bounds

About an algorithm A

- Upper bound: $T_A(n) = O(g)$, i.e. A never costs more than this
- Lower bound: $T_A(n) = \Omega(g)$, i.e. A really does cost this on some inputs (worst case)
- Proved by analysing **this one algorithm**

About a problem P (in a model)

- Upper bound: **some** algorithm solves P in $O(g)$
- Lower bound: **every** algorithm for P in the model needs $\Omega(g)$
- Tight when they match. Example: comparison sorting is $\Theta(n \log n)$.

WHY THIS MATTERS

An upper bound is an algorithm. A lower bound is a **theorem about all algorithms**, including ones nobody has invented yet.

Once they match, you stop searching for a faster algorithm *in that model*. To go faster you have to change the model or the assumptions.

Asymptotics vs. practice

- **Constants and lower-order terms**: insertion sort beats merge sort below roughly 16–64 elements, so library sorts switch to it for small subarrays.
- **Memory hierarchy**: a cache miss costs about 100 comparisons. Cache-friendly $\Theta(n \log n)$ code can beat pointer-chasing $\Theta(n)$ code.
- **Input distributions**: real data is often nearly sorted, skewed or adversarial.
- **Galactic algorithms**: asymptotically best, never practical (e.g. fast matrix multiplication with $\omega < 2.372$).
- **Which cost?** Comparisons, time, memory, I/Os, energy. These can disagree.

LOOK AHEAD (BLOCK C DATA)

`std::sort` makes **~30–40% more comparisons** than merge sort on random input, yet it is the default in practice. Why might that be sensible?

Quick check: true, false, or meaningless?

1. "Merge sort is $O(n^2)$."
2. "Quicksort is $O(n \log n)$."
3. "Insertion sort is $\Omega(n^2)$."
4. "A $\Theta(n \log n)$ worst-case algorithm is always faster than a $\Theta(n^2)$ worst-case one."
5. "Counting sort breaks the $\Omega(n \log n)$ sorting lower bound."
6. "Hash table lookup is $O(1)$."
7. "Dynamic-array append is $O(1)$ on average."

> **3 MIN, INDIVIDUALLY**
> For each: T / F / needs qualification. If it needs one, write the qualification.

BLOCK B · 01:10–02:00

Proving algorithms correct

Invariants · induction · contradiction · counterexamples · exchange arguments (preview)

What a correctness proof must establish

1. **Specification**: precondition (valid inputs) and postcondition (required output). Without a spec there is nothing to prove.
2. **Partial correctness**: *if* the algorithm terminates on a valid input, the output satisfies the postcondition.
3. **Termination**: it does terminate, usually via a *variant*, a non-negative integer that strictly decreases on every iteration or recursive call.

Total correctness = 2 + 3.

TOOLBOX

Loops	loop invariant + variant
Recursion	(strong) induction on input size
Optimality	contradiction; exchange / cut arguments
Refutation	counterexample

Testing can show the **presence** of bugs, never their absence. A proof covers *all* inputs that satisfy the precondition.

Loop invariants: binary search done right

Spec. Pre: $A[0..n-1]$ sorted non-decreasingly. Post: return the smallest $i \in [0, n]$ with $A[i] \geq x$ (or n if none).

Invariant I: $0 \leq lo \leq hi \leq n$, $A[j] < x$ for all $j < lo$, and $A[j] \geq x$ for all $j \geq hi$.

Variant: $hi - lo$.

```
1 LOWER-BOUND(A, n, x)
2   lo = 0 ; hi = n
3   while lo < hi
4       mid = lo + ⌊(hi − lo) / 2⌋
5       if A[mid] < x
6           lo = mid + 1
7       else
8           hi = mid
9   return lo
```

Initialization. $lo = 0, hi = n$: both "for all" ranges are empty, so I holds.

Maintenance. $lo \leq mid < hi$. If $A[mid] < x$, sortedness gives $A[j] < x$ for all $j \leq mid$, so $lo = mid + 1$ preserves I. Otherwise $A[j] \geq x$ for all $j \geq mid$, so $hi = mid$ does.

Termination. $hi - lo$ strictly decreases, because $mid < hi$ and $mid + 1 > lo$. At exit $lo = hi$; by I, lo is the smallest index with $A[lo] \geq x$.

Induction for recursive algorithms

```
1 POWER(a, n)          ▷ n ≥ 0 integer
2   if n == 0
3       return 1
4   h = POWER(a, ⌊n/2⌋)
5   if n is even
6       return h · h
7   else
8       return h · h · a
```

Claim. For all $n \geq 0$: $\mathrm{POWER}(a, n) = a^n$.

Proof (strong induction on n). Base $n = 0$: returns $1 = a^0$. Step: for $n \geq 1$, $\lfloor n/2 \rfloor < n$, so by hypothesis $h = a^{\lfloor n/2 \rfloor}$. If $n = 2k$: $h^2 = a^{2k}$. If $n = 2k + 1$: $h^2 a = a^{2k+1}$.

Termination is part of the same induction: every call is on a strictly smaller non-negative integer.

Cost. $T(n) = T(\lfloor n/2 \rfloor) + O(1) \Rightarrow O(\log n)$ multiplications. How many *bit operations*?

Contradiction: finding the maximum needs $n - 1$ comparisons

Claim. Every comparison-based algorithm that always returns the maximum of n distinct keys makes at least $n - 1$ comparisons in the worst case.

Proof. Call a key a *loser* once it has been smaller in some comparison. Suppose the algorithm stops after fewer than $n - 1$ comparisons. Each comparison creates at most one new loser, so at least two keys, u and v, have never lost. The algorithm outputs one of them, say u. Now change the input by raising v above every other key. Every comparison involving v had v as the winner, so all comparison outcomes are unchanged, the algorithm behaves identically, and it outputs u, which is wrong. Contradiction. ■

PATTERN

Assume a *better* algorithm exists → build an input on which it is fooled.

This is an **adversary argument**. The same idea proves lower bounds in Block C.

Counterexamples: the fastest way to kill a claim

Claim A: greedy coin change is optimal

Coins $\{1, 3, 4\}$, amount 6. Greedy: $4 + 1 + 1$ = 3 coins. Optimal: $3 + 3$ = 2 coins. ✗

Greedy *is* optimal for some coin systems (e.g. $\{1, 5, 10, 25\}$). Knowing *when* is L08 material.

Claim B: shortest-first maximises the number of non-overlapping intervals

$[0, 5), [4, 7), [6, 11)$: shortest-first picks $[4, 7)$ only. Optimal: $[0, 5), [6, 11)$. ✗

HOW TO FIND COUNTEREXAMPLES SYSTEMATICALLY

1. Try tiny inputs ($n \leq 4$) and extreme structure: all equal, sorted, reversed, one huge value.
2. Identify the "decision" the rule makes and construct an input where that decision blocks a better one.
3. Mechanise: brute force vs. candidate on thousands of random small inputs (*differential testing*).

Preview: exchange arguments

Rule: repeatedly pick the compatible interval that **finishes earliest**.

Exchange lemma. Let g be the interval with the earliest finish time. Some optimal solution contains g.

Proof. Take any optimal solution O, sorted by finish time, with first interval o_1. Since $f(g) \leq f(o_1)$, replacing o_1 by g conflicts with nothing: every other interval in O starts after $f(o_1) \geq f(g)$. The new solution is feasible and has $|O|$ intervals, so it is optimal. Then apply induction to the intervals that start after $f(g)$.

TEMPLATE (L08, L09)

1. Take an arbitrary optimal solution O.
2. If it differs from greedy, **exchange** one piece towards greedy's choice.
3. Show the result is still feasible and no worse.
4. Repeat or induct: greedy's solution is optimal.

Cut property for MSTs (L09) is the same argument.

Exercise B: find the flaw

Spec. A sorted, $A[0] \le x$. Return the largest i with $A[i] \le x$.

```
1 LAST-LE(A, n, x)
2   lo = 0 ; hi = n − 1
3   while lo < hi
4       mid = ⌊(lo + hi) / 2⌋
5       if A[mid] ≤ x
6           lo = mid
7       else
8           hi = mid − 1
9   return lo
```

A STUDENT'S "PROOF"

Invariant: $A[lo] \le x$, and every $j > hi$ has $A[j] > x$. Both branches preserve it. The interval $[lo, hi]$ shrinks every iteration, so the loop terminates, and at exit $lo = hi$ is the answer.

GROUPS OF 3 · 8 MIN

(1) Which sentence is false? (2) Give an input that breaks the algorithm. (3) Fix it with a minimal change and re-prove termination.

Common ways correctness proofs fail

Failure	Symptom	Remedy
No specification	"It sorts the array" (stable? duplicates? in place?)	Write pre/postconditions first
Invariant too weak	Cannot derive the postcondition at exit	Ask: what do we know about the *discarded* part?
Invariant not actually maintained	Proved on an example, not in general	Consider every branch and boundary
Termination asserted, not proved	Off-by-one infinite loops (Exercise B)	Name a variant; check floors/ceilings
Circular induction	Hypothesis applied to an instance that is not smaller	State the measure that decreases
"Clearly" / "obviously"	The hard step is skipped	If it is obvious, it takes one line: write it

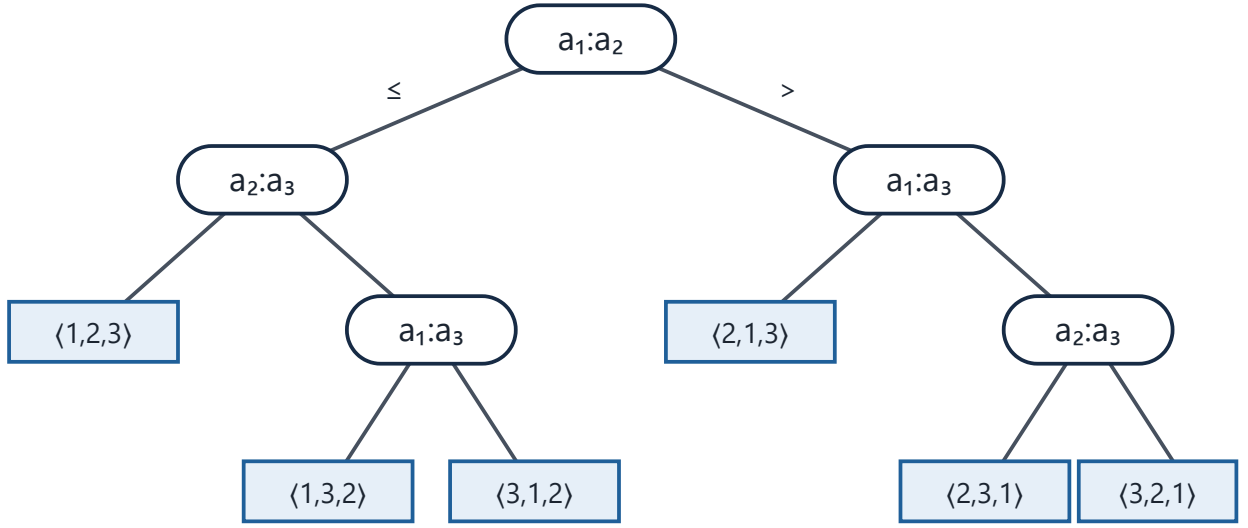
BLOCK C · 02:10–02:55

Lower bounds: proving that nothing is faster

Decision trees · $\Omega(n \log n)$ · what the bound does not say · experiment

The comparison decision tree

- Fix n. Any deterministic comparison sort, run on inputs of size n, is a **binary tree**: internal node = a comparison $a_i \le a_j$?; leaf = an output permutation.
- One execution = one root-to-leaf path. Worst-case comparisons = **height**.
- Correctness forces every one of the $n!$ orderings to reach a leaf labelled with the right permutation, so there are $\ge n!$ leaves.



Insertion sort on n = 3: 6 leaves, height 3 = $\lceil \log_2 3! \rceil$

Theorem: comparison sorting needs $\Omega(n \log n)$

THEOREM

Every comparison sort makes at least $\lceil \log_2 n! \rceil = n \log_2 n - n \log_2 e + O(\log n)$ comparisons in the worst case, and at least $\log_2 n!$ on average over a uniformly random permutation.

PROOF

A binary tree of height h has at most 2^h leaves. We need $2^h \geq n!$, so $h \geq \log_2 n!$. And

$$\log_2 n! = \sum_{i=1}^{n} \log_2 i \;\geq\; \sum_{i=\lceil n/2 \rceil}^{n} \log_2 i \;\geq\; \frac{n}{2} \log_2 \frac{n}{2} \;=\; \Omega(n \log n).$$

Average case: among binary trees with L leaves, average leaf depth $\geq \log_2 L$ (proof by induction or by convexity; balanced trees are best).

■

Vendor claim 1: $\lceil \log_2 20! \rceil = 62 > 60 = 3 \cdot 20$, so "$3n$ comparisons on any input" is impossible for every $n \geq 20$. For $n = 10^6$: $\log_2 n! \approx 1.85 \times 10^7 \gg 3 \times 10^6$.

What the lower bound does *not* say

Counting sort: keys in $[0, K)$

Count occurrences, take prefix sums, place each key. $\Theta(n + K)$ time and space. **It never compares two keys**: it uses a key as an *array index*.

Radix sort: b-bit keys, digits of r bits

$\lceil b/r \rceil$ stable counting-sort passes: $\Theta\big(\frac{b}{r}(n + 2^r)\big)$. With $r \approx \log_2 n$: $\Theta\big(\frac{b\,n}{\log n}\big)$, i.e. $\Theta(n)$ when $b = O(\log n)$.

NO CONTRADICTION

The theorem is about the **comparison model**. Counting and radix sort run in the **word-RAM** and **assume** integer keys from a bounded universe.

THE PRICE OF THE ASSUMPTION

Counting sort pays $\Theta(K)$ memory and time. Radix sort's cost depends on the key length b. For arbitrary comparable objects (strings, tuples with a custom order), you are back in the comparison model.

Experiment 1: how close are real sorts to $\lceil \log_2 n! \rceil$?

n	$\lceil \log_2 n! \rceil$	merge sort	std::sort	insertion
16	45	46	72	67
256	1,684	1,731	2,445	16,421
4,096	43,251	43,983	60,125	4,186,229
65,536	954,037	965,692	1,261,749	—

INTERPRET

- Merge sort is within ~1–3% of the bound. Why so close?
- Insertion sort $\approx n^2/4$. Why exactly that?
- Does any column contradict the theorem?

Random permutations, mean of 5 runs; `Student_Resources/comparison_counts.cpp`, g++ 15.2 -O2.

Experiment 2: when does the extra assumption pay off?

$n = 2{,}000{,}000$ keys uniform in $[0, K)$; median of 5 runs (ms)		
K	counting sort	std::sort
256	5.5	63.7
65,536	12.9	107.2
2,000,000	30.0	127.0
64,000,000	**199.8**	**124.0**
256,000,000	**719.3**	**128.9**

DISCUSS · 3 MIN

1. Predict the crossover from the formulas $\Theta(n + K)$ vs $\Theta(n \log n)$.
2. Why is counting sort still 4× faster at $K = n$?
3. Name one threat to validity of this table.

Exercise C: lower bounds for other problems

1. **Searching.** Prove that any comparison-based search in a sorted array of n keys needs $\geq \lceil \log_2(n+1) \rceil$ comparisons in the worst case.
2. **Merging.** Two sorted lists of n keys each. (a) Give the decision-tree bound. (b) Show that an adversary forces $2n - 1$ comparisons. *Hint:* interleave the lists.
3. **Reduction.** Prove that building a binary search tree from n unsorted keys needs $\Omega(n \log n)$ comparisons.
4. **Too good to be true.** A comparison-based priority queue with $O(1)$ *insert* and $O(1)$ *extract-min*, both worst case. Possible?

> **GROUPS · 10 MIN**
>
> Do 1 and 3 first. Then 4. Then 2 if time permits.

Synthesis: what to carry forward

1. A complexity claim = **model** + **input size** + **cost measure** + **kind of guarantee** + bound.
2. $O/\Omega/\Theta$ compare functions. Worst/best/average/expected/amortized choose the function. Keep the two axes separate.
3. Correctness = specification + partial correctness + termination. Invariants for loops, induction for recursion, exchange arguments for greedy.
4. A problem lower bound is a theorem about **all** algorithms in a model: adversaries, decision trees, reductions.
5. Beating a lower bound always means changing the model or adding an assumption, and assumptions have costs you can measure.

NEXT WEEK · L02 (SAT 3 OCT)

Divide-and-conquer, recurrences (Master theorem and where it fails), and **selection in linear time**: finding the median without sorting, deterministically.

Read: CLRS 4e Ch. 4 (4.3–4.6), Ch. 9. **PS1** is released next Saturday.